Proxmox VE 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 6
1.1 Proxmox VE とは 6
1.2 なぜ Proxmox VE を選ぶのか 6
1.2.1 他の仮想化プラットフォームとの比較 6
1.3 このガイドについて 7
1.4 対象読者 7
1.5 前提知識 7
2 インストール 8
2.1 システム要件 8
2.1.1 最小要件 8
2.1.2 推奨要件 8
2.2 ISO イメージのダウンロード 8
2.3 インストールメディアの作成 8
2.3.1 USB メモリへの書き込み 8
2.4 インストール手順 9
2.4.1 BIOS/UEFI の設定 9
2.4.2 インストーラーの起動 9
2.4.3 ディスクの設定 9
2.4.4 ネットワークの設定 9
2.4.5 パスワードとメールの設定 9
2.4.6 インストールの完了 9
2.5 初期設定 10
2.5.1 Web UI へのアクセス 10
2.5.2 サブスクリプションの無効化（任意） 10
2.5.3 タイムゾーンと NTP の確認 10
3 基本操作 11
3.1 Web UI の構成 11
3.2 ノードの管理 11
3.2.1 ノード情報の確認 11
3.2.2 シェルアクセス 11
3.3 コマンドラインツール 11
3.3.1 qm（仮想マシン管理） 11
3.3.2 pct（コンテナ管理） 12
3.3.3 pvesm（ストレージ管理） 12
3.3.4 pvesh（API シェル） 12
3.4 ISO イメージのアップロード 13
3.4.1 Web UI から 13
3.4.2 コマンドラインから 13
3.5 テンプレートのダウンロード 13

3.5.1 Web UI から . . . 13
3.5.2 コマンドラインから . . . 13
4 仮想マシン . . . 14
4.1 仮想マシンの作成 . . . 14
4.1.1 Web UI から作成 . . . 14
4.1.2 コマンドラインから作成 . . . 15
4.2 VirtIO ドライバー . . . 15
4.2.1 Linux ゲスト . . . 15
4.2.2 Windows ゲスト . . . 15
4.3 QEMU Guest Agent . . . 15
4.3.1 有効化と導入 . . . 15
4.4 スナップショット . . . 16
4.5 クローン . . . 16
4.5.1 フルクローン . . . 16
4.5.2 リンクドクローン . . . 16
4.6 テンプレート . . . 17
5 コンテナ（LXC） . . . 18
5.1 LXC コンテナとは . . . 18
5.2 コンテナの作成 . . . 18
5.2.1 Web UI から作成 . . . 18
5.2.2 コマンドラインから作成 . . . 19
5.3 コンテナの管理 . . . 19
5.3.1 基本操作 . . . 19
5.3.2 リソースの変更 . . . 19
5.4 特権コンテナと非特権コンテナ . . . 20
5.4.1 非特権コンテナ（推奨） . . . 20
5.4.2 特権コンテナ . . . 20
5.5 マウントポイント . . . 20
5.6 コンテナのバックアップと復元 . . . 20
6 ストレージ . . . 21
6.1 ストレージの概念 . . . 21
6.1.1 コンテンツタイプ . . . 21
6.2 ストレージの種類 . . . 21
6.2.1 ローカルストレージ . . . 21
6.2.2 共有ストレージ . . . 22
6.3 ストレージの管理 . . . 23
6.4 推奨構成例 . . . 23
6.4.1 単一ノード構成 . . . 23
6.4.2 クラスタ構成 . . . 23
7 ネットワーク . . . 24
7.1 ネットワークの基本構成 . . . 24
7.2 ブリッジの設定 . . . 24

7.2.1 Web UI から ... 24
7.2.2 設定ファイル ... 24
7.3 VLAN ... 25
7.3.1 VLAN Aware ブリッジ ... 25
7.4 ボンディング ... 25
7.5 ファイアウォール ... 26
7.5.1 ファイアウォールの有効化 ... 26
7.5.2 ルールの追加 ... 26
7.5.3 セキュリティグループ ... 26
7.6 SDN（Software Defined Networking） ... 27
7.6.1 SDN の有効化 ... 27
8 バックアップとリストア ... 28
8.1 バックアップの種類 ... 28
8.2 バックアップの実行 ... 28
8.2.1 Web UI から ... 28
8.2.2 コマンドラインから ... 28
8.2.3 圧縮方式 ... 28
8.3 スケジュールバックアップ ... 28
8.3.1 Web UI から設定 ... 28
8.3.2 設定ファイル ... 29
8.4 リストア（復元） ... 29
8.4.1 Web UI から ... 29
8.4.2 コマンドラインから ... 29
8.5 Proxmox Backup Server（PBS） ... 29
8.5.1 主な特徴 ... 29
8.5.2 PBS との連携設定 ... 30
8.6 バックアップの保持ポリシー ... 30
9 クラスタ ... 31
9.1 クラスタとは ... 31
9.1.1 クラスタの利点 ... 31
9.1.2 クラスタの要件 ... 31
9.2 クラスタの構築 ... 31
9.2.1 最初のノードでクラスタを作成 ... 31
9.2.2 他のノードの参加 ... 31
9.2.3 クラスタの確認 ... 32
9.3 ライブマイグレーション ... 32
9.3.1 前提条件 ... 32
9.3.2 マイグレーションの実行 ... 32
9.4 高可用性（HA） ... 32
9.4.1 HA の設定 ... 32
9.4.2 HA グループ ... 33
9.4.3 フェンシング ... 33

9.5 2 ノードクラスタ 33
9.6 クラスタからのノード削除 33
10 Tips 34
10.1 Cloud-Init 34
10.2 PCI パススルー 34
10.2.1 IOMMU の有効化 34
10.2.2 VFIO モジュールの設定 34
10.2.3 デバイスの割り当て 35
10.3 REST API の活用 35
10.4 Terraform との連携 35
10.5 トラブルシューティング 36
10.5.1 よくある問題と対処法 36
10.6 便利なコマンド集 37

1 はじめに

1.1 Proxmox VE とは

Proxmox VE（Virtual Environment）は、オープンソースの仮想化管理プラットフォームです。Debian GNU/Linux をベースに、KVM（カーネルベース仮想マシン）と LXC（Linux コンテナ）の両方を統合管理できる環境を提供します。

主な特徴は以下の通りです：

- **オープンソース**：AGPLv3 ライセンスで無償利用可能（商用サポートは有償）
- **Web UI**：ブラウザベースの管理インターフェースで直感的に操作できる
- **KVM 仮想化**：Windows、Linux、BSD など多様な OS を完全仮想化で動作
- **LXC コンテナ**：軽量なコンテナ型仮想化でリソース効率を最大化
- **クラスタ対応**：複数ノードをクラスタ化し、高可用性（HA）を実現
- **統合ストレージ**：ZFS、Ceph、NFS、iSCSI など多様なストレージバックエンドをサポート
- **バックアップ**：統合バックアップツール（vzdump）と Proxmox Backup Server 連携
- **REST API**：自動化やスクリプトによる管理が可能

1.2 なぜ Proxmox VE を選ぶのか

1.2.1 他の仮想化プラットフォームとの比較

項目	Proxmox VE	VMware ESXi	Hyper-V
ライセンス	オープンソース（無償）	有償（無償版は制限あり）	Windows Server に付属
ハイパーバイザー	KVM + LXC	独自（vmkernel）	独自
管理 UI	Web UI（標準）	vSphere Client	Hyper-V マネージャー
コンテナ	LXC 対応	非対応	非対応
ストレージ	ZFS, Ceph 統合	vSAN（有償）	S2D
コスト	低い	高い	中程度

Proxmox VE は特に以下のようなユースケースに適しています：

- ホームラボでの学習・検証環境の構築
- 中小企業のサーバー仮想化
- 開発・テスト環境の構築
- プライベートクラウドの基盤

1.3 このガイドについて

本書は Proxmox VE を初めて使う方を対象に、インストールから基本操作、 仮想マシン・コンテナの管理、ストレージ・ネットワークの設定、 クラスタ構成までを網羅的に解説する入門書です。

1.4 対象読者

- Linux の基本操作に慣れている方
- 仮想化技術に興味がある方
- 自宅サーバーやホームラボを構築したい方
- VMware からの移行を検討している方

1.5 前提知識

- 基本的な Linux コマンド操作（`cd`、`ls`、`ip` など）
- ネットワークの基礎知識（IP アドレス、サブネット、VLAN など）
- 仮想化の基本概念（仮想マシン、ハイパーバイザーなど）の理解があると望ましい

2 インストール

2.1 システム要件

Proxmox VE を動作させるためのハードウェア要件は以下の通りです。

2.1.1 最小要件

- **CPU**：64bit（Intel EMT64 または AMD64）、Intel VT/AMD-V 対応
- **メモリ**：2 GB 以上（推奨 4 GB 以上、ゲスト OS 分は別途必要）
- **ストレージ**：32 GB 以上（推奨 SSD）
- **ネットワーク**：1 つ以上の NIC

2.1.2 推奨要件

- **CPU**：マルチコア（4 コア以上）、VT-d/AMD-Vi 対応（PCI パススルー用）
- **メモリ**：16 GB 以上（ECC メモリ推奨）
- **ストレージ**：OS 用 SSD + データ用ストレージ（ZFS 利用時は ECC メモリ推奨）
- **ネットワーク**：2 つ以上の NIC（管理用と VM 用で分離）

2.2 ISO イメージのダウンロード

Proxmox VE の ISO イメージは公式サイトからダウンロードします。

1. ブラウザで `https://www.proxmox.com/en/downloads` にアクセス
2. 「Proxmox VE ISO Installer」の最新版をダウンロード
3. ダウンロード後、チェックサムを検証

```
# SHA256 チェックサムの検証
sha256sum proxmox-ve_8.x-x.iso
```

2.3 インストールメディアの作成

2.3.1 USB メモリへの書き込み

```
# Linux の場合
dd bs=1M conv=fdatasync if=proxmox-ve_8.x-x.iso of=/dev/sdX

# macOS の場合
sudo dd if=proxmox-ve_8.x-x.iso of=/dev/rdiskN bs=1m
```

Windows の場合は Rufus や Etcher などのツールを使用してください。

2.4 インストール手順

2.4.1 BIOS/UEFI の設定

インストール前に以下の設定を確認・変更してください：

1. **Intel VT-x / AMD-V**：有効化（仮想化支援）
2. **Intel VT-d / AMD-Vi**：有効化（PCI パススルー用、任意）
3. **ブート順序**：USB を最優先に設定

2.4.2 インストーラーの起動

1. USB メモリからブート
2.「Install Proxmox VE (Graphical)」を選択
3. EULA に同意

2.4.3 ディスクの設定

ターゲットディスクを選択します。「Options」ボタンで詳細設定が可能です：

- **ext4**：一般的なファイルシステム（シンプル）
- **xfs**：高パフォーマンス
- **ZFS（RAID0/1/10/RAIDZ）**：データ保護・スナップショット機能（推奨）

ZFS を選択する場合は複数ディスクの構成を設定できます。

2.4.4 ネットワークの設定

以下を入力します：

- **Management Interface**：管理用 NIC を選択
- **Hostname (FQDN)**：`pve.local` など
- **IP Address**：固定 IP を設定（例：`192.168.1.100/24`）
- **Gateway**：デフォルトゲートウェイ
- **DNS Server**：DNS サーバーのアドレス

2.4.5 パスワードとメールの設定

- **Password**：root ユーザーのパスワード
- **E-mail**：管理者のメールアドレス（通知用）

2.4.6 インストールの完了

設定を確認して「Install」をクリックすると、インストールが開始されます。完了後、再起動するとコンソールに Web UI の URL が表示されます。

2.5 初期設定

2.5.1 Web UI へのアクセス

ブラウザで以下の URL にアクセスします：

```
https://<IPアドレス>:8006
```

自己署名証明書のため、ブラウザの警告が表示されますが続行してください。ログイン情報は以下の通りです：

- **ユーザー名**：`root`
- **レルム**：`Linux PAM standard authentication`
- **パスワード**：インストール時に設定したパスワード

2.5.2 サブスクリプションの無効化（任意）

無償で利用する場合、エンタープライズリポジトリを無効化し、no-subscription リポジトリを有効化します。

```
# エンタープライズリポジトリを無効化
mv /etc/apt/sources.list.d/pve-enterprise.list \
   /etc/apt/sources.list.d/pve-enterprise.list.bak

# no-subscription リポジトリを追加
echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" \
  > /etc/apt/sources.list.d/pve-no-subscription.list

# パッケージの更新
apt update && apt full-upgrade -y
```

2.5.3 タイムゾーンと NTP の確認

```
# タイムゾーンの確認・設定
timedatectl set-timezone Asia/Tokyo

# NTP の確認（chrony が標準）
systemctl status chronyd
```

3 基本操作

3.1 Web UI の構成

Proxmox VE の Web UI は以下の主要エリアで構成されています：

- **ヘッダーバー**：ユーザー情報、ドキュメントリンク、ログアウト
- **リソースツリー（左側）**：データセンター、ノード、VM、コンテナの階層表示
- **コンテンツパネル（中央）**：選択した項目の詳細情報と操作
- **タスクログ（下部）**：実行中・完了済みタスクの履歴

3.2 ノードの管理

3.2.1 ノード情報の確認

リソースツリーでノードを選択すると、以下の情報を確認できます：

- **Summary**：CPU、メモリ、ストレージの使用状況
- **Notes**：ノードに関するメモ
- **Shell**：Web ベースのコンソール（noVNC / xterm.js）
- **System**：ネットワーク、DNS、時刻、Syslog の設定
- **Updates**：パッケージの更新状態

3.2.2 シェルアクセス

Web UI から直接シェルにアクセスする方法は 2 つあります：

1. **Web UI のシェル**：ノードを選択 → 「Shell」ボタン
2. **SSH 接続**：

```
ssh root@<IPアドレス>
```

3.3 コマンドラインツール

Proxmox VE には専用の CLI ツールが用意されています。

3.3.1 qm（仮想マシン管理）

```
# VM 一覧
qm list

# VM の起動
qm start <vmid>

# VM の停止
```

```
qm stop <vmid>

# VM のシャットダウン（ゲスト OS 経由）
qm shutdown <vmid>

# VM の設定確認
qm config <vmid>

# VM の作成
qm create <vmid> --name <名前> --memory 2048 --cores 2
```

3.3.2 pct（コンテナ管理）

```
# コンテナ一覧
pct list

# コンテナの起動
pct start <ctid>

# コンテナの停止
pct stop <ctid>

# コンテナの設定確認
pct config <ctid>

# コンテナにログイン
pct enter <ctid>
```

3.3.3 pvesm（ストレージ管理）

```
# ストレージ一覧
pvesm status

# ストレージの追加（NFS の例）
pvesm add nfs <ストレージ名> --server <サーバー> --export <パス>
```

3.3.4 pvesh（API シェル）

```
# ノード情報の取得
pvesh get /nodes

# VM 一覧の取得
pvesh get /nodes/<ノード名>/qemu

# リソースの取得
pvesh get /cluster/resources
```

3.4 ISO イメージのアップロード

VM にインストールする OS の ISO イメージをアップロードします。

3.4.1 Web UI から

1. データセンター → ストレージ → `local` を選択
2. 「ISO Images」タブを選択
3. 「Upload」ボタンからファイルを選択
4. アップロードが完了するまで待機

3.4.2 コマンドラインから

```
# wget で直接ダウンロード
cd /var/lib/vz/template/iso/
wget https://example.com/os-image.iso
```

3.5 テンプレートのダウンロード

LXC コンテナ用のテンプレートをダウンロードします。

3.5.1 Web UI から

1. ストレージ → `local` → 「CT Templates」タブ
2. 「Templates」ボタンをクリック
3. 使用したいテンプレートを選択してダウンロード

3.5.2 コマンドラインから

```
# テンプレート一覧の更新
pveam update

# 利用可能なテンプレートの表示
pveam available

# テンプレートのダウンロード
pveam download local debian-12-standard_12.2-1_amd64.tar.zst
```

4 仮想マシン

4.1 仮想マシンの作成

4.1.1 Web UI から作成

1. 右上の「Create VM」ボタンをクリック
2. 各タブで設定を行う：

General
- **Node**：VM を配置するノード
- **VM ID**：一意の番号（デフォルトで自動採番）
- **Name**：VM の名前

OS
- **ISO image**：インストールに使用する ISO を選択
- **Guest OS Type**：Linux / Windows / Other

System
- **Machine**：q35（推奨）または i440fx
- **BIOS**：OVMF（UEFI）または SeaBIOS
- **SCSI Controller**：VirtIO SCSI single（推奨）
- **Qemu Agent**：有効化推奨

Disks
- **Bus/Device**：VirtIO Block または SCSI
- **Storage**：ディスクの保存先
- **Disk size**：ディスクサイズ（GiB）
- **Discard**：SSD 使用時はチェック（TRIM サポート）

CPU
- **Cores**：割り当てるコア数
- **Type**：host（最高性能）または x86-64-v2-AES（互換性重視）

Memory
- **Memory (MiB)**：割り当てるメモリ量
- **Ballooning Device**：動的メモリ管理（有効化推奨）

Network
- **Bridge**：`vmbr0`（デフォルト）
- **Model**：VirtIO（推奨）
- **VLAN Tag**：必要に応じて設定

4.1.2 コマンドラインから作成

```
# 基本的な VM 作成
qm create 100 \
  --name my-vm \
  --memory 4096 \
  --cores 2 \
  --scsihw virtio-scsi-single \
  --scsi0 local-lvm:32 \
  --ide2 local:iso/debian-12.iso,media=cdrom \
  --net0 virtio,bridge=vmbr0 \
  --boot order=ide2

# VM の起動
qm start 100
```

4.2 VirtIO ドライバー

4.2.1 Linux ゲスト

Linux カーネルには VirtIO ドライバーが標準で含まれているため、 追加のインストールは不要です。

4.2.2 Windows ゲスト

Windows では VirtIO ドライバーを別途インストールする必要があります。

1. VirtIO ISO をダウンロード（`https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-win.iso`
2. VM に第二の CD ドライブとして VirtIO ISO をマウント
3. Windows インストール時にドライバーを読み込み、またはインストール後に実行

```
# VirtIO ISO を第二ドライブに追加
qm set 100 --ide0 local:iso/virtio-win.iso,media=cdrom
```

4.3 QEMU Guest Agent

QEMU Guest Agent を導入すると、ホストからゲスト OS を適切に管理できます。

4.3.1 有効化と導入

```
# VM 側で Guest Agent を有効化
qm set <vmid> --agent enabled=1

# Linux ゲストへのインストール
apt install qemu-guest-agent    # Debian/Ubuntu
```

```
dnf install qemu-guest-agent    # RHEL/Fedora

systemctl enable --now qemu-guest-agent
```

Guest Agent を有効化すると以下が可能になります：

- IP アドレスの正確な表示
- ファイルシステムの静止（freeze/thaw）によるバックアップの整合性向上
- ゲスト OS の適切なシャットダウン

4.4 スナップショット

スナップショットは VM の状態を保存し、後から復元できる機能です。

```
# スナップショットの作成
qm snapshot <vmid> <スナップショット名> --description "説明"

# スナップショット一覧
qm listsnapshot <vmid>

# スナップショットへのロールバック
qm rollback <vmid> <スナップショット名>

# スナップショットの削除
qm delsnapshot <vmid> <スナップショット名>
```

4.5 クローン

既存の VM を複製する方法は 2 種類あります。

4.5.1 フルクローン

ディスクを完全にコピーします。元の VM とは独立した VM が作成されます。

```
qm clone <元vmid> <新vmid> --name <名前> --full
```

4.5.2 リンクドクローン

ベースイメージを共有し、差分のみ保存します。ディスク使用量を節約できます。

```
# 先にテンプレートに変換
qm template <vmid>

# リンクドクローンの作成
qm clone <元vmid> <新vmid> --name <名前>
```

4.6 テンプレート

頻繁に同じ構成の VM を作成する場合、テンプレート化が便利です。

1. VM を通常通り作成・設定
2. OS のインストールと初期設定を完了
3. `cloud-init` パッケージをインストール（任意）
4. VM をシャットダウン
5. テンプレートに変換

```
qm template <vmid>
```

テンプレートからは「Clone」で新しい VM を作成できます。

5 コンテナ（LXC）

5.1 LXC コンテナとは

LXC（Linux Containers）は、Linux カーネルの機能（namespaces、cgroups）を利用した OS レベルの仮想化技術です。仮想マシンと比較して以下の利点があります：

- **軽量**：カーネルを共有するためオーバーヘッドが少ない
- **高速起動**：数秒で起動
- **低リソース消費**：メモリ・CPU の使用効率が良い
- **ほぼネイティブな性能**：ハードウェアエミュレーションが不要

一方で以下の制約があります：

- Linux カーネルを共有するため、Linux OS のみ動作可能
- カーネルモジュールのロードなど、一部の操作が制限される
- Windows や BSD は動作不可

5.2 コンテナの作成

5.2.1 Web UI から作成

1. 右上の「Create CT」ボタンをクリック
2. 各タブで設定を行う：

General

- **Node**：コンテナを配置するノード
- **CT ID**：一意の番号
- **Hostname**：ホスト名
- **Password**：root パスワード
- **SSH Public Key**：SSH 公開鍵（任意）

Template

- ダウンロード済みのテンプレートを選択

Root Disk

- **Storage**：ディスクの保存先
- **Disk size**：ルートディスクのサイズ（GiB）

CPU

- **Cores**：割り当てるコア数

Memory

- **Memory (MiB)**：メモリ量
- **Swap (MiB)**：スワップ量

Network

- **Bridge**：`vmbr0`
- **IPv4/IPv6**：DHCP または固定 IP

DNS
- **DNS domain**：ドメイン名
- **DNS servers**：DNS サーバー

5.2.2 コマンドラインから作成

```
# コンテナの作成
pct create 200 local:vztmpl/debian-12-standard_12.2-1_amd64.tar.zst \
  --hostname my-container \
  --memory 1024 \
  --swap 512 \
  --cores 2 \
  --rootfs local-lvm:8 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp \
  --password

# コンテナの起動
pct start 200
```

5.3 コンテナの管理

5.3.1 基本操作

```
# コンテナ一覧
pct list

# コンテナの起動・停止
pct start <ctid>
pct stop <ctid>
pct shutdown <ctid>

# コンテナに入る
pct enter <ctid>

# コンテナ内でコマンド実行
pct exec <ctid> -- apt update
```

5.3.2 リソースの変更

コンテナはホットプラグに対応しており、稼働中にリソースを変更できます。

```
# メモリの変更
pct set <ctid> --memory 2048
```

```
# CPU コア数の変更
pct set <ctid> --cores 4

# ディスクのリサイズ
pct resize <ctid> rootfs +5G
```

5.4 特権コンテナと非特権コンテナ

5.4.1 非特権コンテナ（推奨）

デフォルトの設定です。UID/GID がマッピングされ、ホストとは異なるユーザー空間で動作します。セキュリティが高く、通常はこちらを使用します。

5.4.2 特権コンテナ

コンテナ内の root がホストの root と同じ権限を持ちます。 特定のアプリケーション（NFS サーバーなど）で必要な場合にのみ使用します。

```
# 特権コンテナの作成
pct create 201 local:vztmpl/debian-12-standard_12.2-1_amd64.tar.zst \
  --unprivileged 0 \
  --hostname privileged-ct \
  --memory 1024 \
  --rootfs local-lvm:8 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp
```

5.5 マウントポイント

ホストのディレクトリやストレージをコンテナ内にマウントできます。

```
# バインドマウント（ホストのディレクトリを共有）
pct set <ctid> --mp0 /host/path,mp=/container/path

# ストレージからの追加ディスク
pct set <ctid> --mp0 local-lvm:10,mp=/data
```

5.6 コンテナのバックアップと復元

```
# バックアップ
vzdump <ctid> --storage local --compress zstd

# 復元
pct restore <新ctid> /var/lib/vz/dump/vzdump-lxc-<ctid>-*.tar.zst
```

6 ストレージ

6.1 ストレージの概念

Proxmox VE では、ストレージをコンテンツタイプで分類して管理します。

6.1.1 コンテンツタイプ

- **images**：VM のディスクイメージ
- **rootdir**：コンテナのルートディレクトリ
- **iso**：ISO イメージファイル
- **vztmpl**：コンテナテンプレート
- **backup**：バックアップファイル
- **snippets**：スニペット（cloud-init 設定など）

6.2 ストレージの種類

6.2.1 ローカルストレージ

6.2.1.1 LVM

Proxmox VE のデフォルトインストールで使用されるストレージです。

```
# LVM の状態確認
lvs
vgs

# LVM ストレージの追加
pvesm add lvm <名前> --vgname <VG名> --content images,rootdir
```

6.2.1.2 LVM-Thin

シンプロビジョニングに対応した LVM です。スナップショットが可能で、 実際に使用された分だけディスクを消費します。

```
pvesm add lvmthin <名前> --vgname <VG名> --thinpool <プール名> \
  --content images,rootdir
```

6.2.1.3 ZFS

ZFS は高機能なファイルシステム兼ボリュームマネージャーです。

主な特徴：

- **データ整合性**：チェックサムによるデータ破損検出
- **スナップショット**：瞬時に作成、ほぼゼロコスト
- **圧縮**：透過的なデータ圧縮
- **RAID**：ソフトウェア RAID（mirror、raidz、raidz2、raidz3）

- **重複排除**：同一データのデデュプリケーション（メモリ消費大）

```
# ZFS プールの作成
zpool create -f mypool mirror /dev/sda /dev/sdb

# ZFS ストレージの追加
pvesm add zfspool <名前> --pool <プール名> --content images,rootdir

# ZFS プールの状態確認
zpool status
zpool list

# ZFS データセットの確認
zfs list
```

6.2.1.4 Directory

通常のディレクトリをストレージとして利用します。

```
# ディレクトリストレージの追加
pvesm add dir <名前> --path /mnt/data --content iso,vztmpl,backup
```

6.2.2 共有ストレージ

6.2.2.1 NFS

ネットワーク経由でファイルを共有するプロトコルです。

```
# NFS ストレージの追加
pvesm add nfs <名前> \
  --server <NFSサーバーIP> \
  --export /share/path \
  --content iso,vztmpl,backup,images
```

6.2.2.2 CIFS/SMB

Windows ファイル共有プロトコルです。

```
pvesm add cifs <名前> \
  --server <サーバーIP> \
  --share <共有名> \
  --username <ユーザー> \
  --content backup,iso,vztmpl
```

6.2.2.3 iSCSI

ブロックデバイスをネットワーク経由で提供するプロトコルです。

```
pvesm add iscsi <名前> \
  --portal <サーバーIP> \
  --target <IQN>
```

6.2.2.4 Ceph (RBD)

分散ストレージシステムです。Proxmox VE に統合されており、 Web UI からクラスタの構築・管理が可能です。

```
pvesm add rbd <名前> \
  --monhost <モニターIP> \
  --pool <プール名> \
  --content images,rootdir
```

6.3 ストレージの管理

```
# ストレージ一覧の確認
pvesm status

# ストレージの詳細
pvesm list <ストレージ名>

# ストレージの削除
pvesm remove <ストレージ名>
```

6.4 推奨構成例

6.4.1 単一ノード構成

- **OS**：ZFS mirror（SSD x 2）
- **VM/CT ディスク**：ZFS（同上またはデータ用プール）
- **ISO・テンプレート**：local ディレクトリ
- **バックアップ**：NFS または外部 HDD

6.4.2 クラスタ構成

- **OS**：各ノードのローカル SSD
- **VM/CT ディスク**：Ceph RBD または共有ストレージ
- **ISO・テンプレート**：NFS（共有）
- **バックアップ**：Proxmox Backup Server

7 ネットワーク

7.1 ネットワークの基本構成

Proxmox VE のネットワークは Linux の標準的なネットワーク機能を使用しています。 デフォルトでは以下のような構成になっています：

- **物理 NIC**（`eno1`、`enp0s3` など）：実際のネットワークインターフェース
- **Linux Bridge**（`vmbr0`）：VM やコンテナが接続する仮想スイッチ

```
物理ネットワーク ── eno1 ── vmbr0 ┬─ VM 100
                                  ├─ VM 101
                                  ├─ CT 200
                                  └─ ホスト管理
```

7.2 ブリッジの設定

7.2.1 Web UI から

1. ノード → System → Network
2. 「Create」→ 「Linux Bridge」を選択
3. 設定項目を入力して適用

7.2.2 設定ファイル

ネットワーク設定は `/etc/network/interfaces` で管理されます。

```
# /etc/network/interfaces の例
auto lo
iface lo inet loopback

# 物理 NIC
auto eno1
iface eno1 inet manual

# メインブリッジ
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
```

設定変更後は以下のコマンドで適用します：

```
ifreload -a
```

7.3 VLAN

VLAN を使用してネットワークを論理的に分離できます。

7.3.1 VLAN Aware ブリッジ

ブリッジを VLAN aware に設定すると、VM ごとに VLAN タグを指定できます。

```
# /etc/network/interfaces
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware yes
    bridge-vids 2-4094
```

VM のネットワーク設定で VLAN Tag を指定：

```
# VM に VLAN 100 を設定
qm set <vmid> --net0 virtio,bridge=vmbr0,tag=100
```

7.4 ボンディング

複数の NIC を束ねて冗長性と帯域幅を向上させます。

```
# /etc/network/interfaces
auto bond0
iface bond0 inet manual
    bond-slaves eno1 eno2
    bond-miimon 100
    bond-mode 802.3ad       # LACP
    bond-xmit-hash-policy layer3+4

auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports bond0
    bridge-stp off
    bridge-fd 0
```

主なボンディングモード：

- **balance-rr (0)**：ラウンドロビン
- **active-backup (1)**：アクティブ/スタンバイ（スイッチ設定不要）
- **802.3ad (4)**：LACP（スイッチ側の設定も必要）

7.5 ファイアウォール

Proxmox VE には統合ファイアウォールが搭載されています。 データセンター、ノード、VM/コンテナの 3 階層で設定できます。

7.5.1 ファイアウォールの有効化

```
# データセンターレベルで有効化
# /etc/pve/firewall/cluster.fw
[OPTIONS]
enable: 1

# VM レベルで有効化
# /etc/pve/firewall/<vmid>.fw
[OPTIONS]
enable: 1
```

7.5.2 ルールの追加

Web UI で「Firewall」タブからルールを追加できます。

```
# 設定ファイルの例（/etc/pve/firewall/<vmid>.fw）
[RULES]
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 22
IN ACCEPT -p tcp -dport 80
IN ACCEPT -p tcp -dport 443
IN DROP
```

7.5.3 セキュリティグループ

よく使うルールセットをグループ化して再利用できます。

```
# /etc/pve/firewall/cluster.fw
[group web-server]
IN ACCEPT -p tcp -dport 80
IN ACCEPT -p tcp -dport 443

[group ssh-access]
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 22
```

7.6 SDN（Software Defined Networking）

Proxmox VE 7 以降では SDN 機能が利用可能です（8.x で正式サポート）。VXLAN や EVPN を使用した高度なネットワーク構成が可能です。

7.6.1 SDN の有効化

Web UI の「Datacenter」→「SDN」から設定できます。

主な概念：

- **Zone**：ネットワーク分離の単位（Simple、VLAN、VXLAN、EVPN）
- **VNet**：仮想ネットワーク
- **Subnet**：VNet 内のサブネット定義

8 バックアップとリストア

8.1 バックアップの種類

Proxmox VE では `vzdump` ツールを使用してバックアップを取得します。3 つのバックアップモードがあります：

- **Snapshot**：稼働中の VM/CT をスナップショットベースでバックアップ（推奨）
- **Suspend**：メモリを保存して一時停止後にバックアップ（ダウンタイムあり）
- **Stop**：完全停止後にバックアップ（最も確実だがダウンタイムあり）

8.2 バックアップの実行

8.2.1 Web UI から

1. VM/CT を選択 → 「Backup」タブ
2. 「Backup now」をクリック
3. モード、圧縮方式、ストレージを選択して実行

8.2.2 コマンドラインから

```
# 基本的なバックアップ
vzdump <vmid> --storage local --compress zstd --mode snapshot

# 全 VM/CT のバックアップ
vzdump --all --storage local --compress zstd --mode snapshot

# 複数指定
vzdump 100 101 200 --storage local --compress zstd
```

8.2.3 圧縮方式

- **none**：無圧縮（高速だがサイズ大）
- **lzo**：軽量圧縮（バランス型）
- **gzip**：標準圧縮
- **zstd**：高圧縮率・高速（推奨）

8.3 スケジュールバックアップ

8.3.1 Web UI から設定

1. データセンター → Backup
2. 「Add」をクリック
3. スケジュール、対象、ストレージ、モードを設定

8.3.2 設定ファイル

スケジュールは `/etc/pve/jobs.cfg` で管理されます。

```
job: vzdump
  schedule daily
  all 1
  enabled 1
  storage local
  compress zstd
  mode snapshot
  mailnotification always
  mailto admin@example.com
```

8.4 リストア（復元）

8.4.1 Web UI から

1. ストレージ → 「Backups」タブ
2. バックアップファイルを選択 → 「Restore」

8.4.2 コマンドラインから

```
# VM のリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 100

# 別の VM ID でリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 105

# コンテナのリストア
pct restore 200 /var/lib/vz/dump/vzdump-lxc-200-*.tar.zst

# ストレージを指定してリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 100 \
  --storage local-lvm
```

8.5 Proxmox Backup Server（PBS）

Proxmox Backup Server は Proxmox VE 専用のバックアップサーバーです。 vzdump よりも高度な機能を提供します。

8.5.1 主な特徴

- **増分バックアップ**：変更されたデータのみを転送・保存
- **重複排除**：データストア全体で重複データを排除
- **暗号化**：クライアントサイドの暗号化
- **整合性検証**：自動的なバックアップの整合性チェック

- **高速リストア**：ファイルレベルのリストアも可能

8.5.2 PBS との連携設定

```
# PBS ストレージの追加
pvesm add pbs <名前> \
  --server <PBSサーバーIP> \
  --username <ユーザー>@pbs \
  --datastore <データストア名> \
  --fingerprint <フィンガープリント>
```

8.6 バックアップの保持ポリシー

バックアップの自動削除ルールを設定できます。

```
# 保持設定の例
vzdump <vmid> --storage local \
  --prune-backups keep-daily=7,keep-weekly=4,keep-monthly=6
```

- **keep-last**：直近 N 個を保持
- **keep-daily**：日次で N 日分
- **keep-weekly**：週次で N 週分
- **keep-monthly**：月次で N ヶ月分
- **keep-yearly**：年次で N 年分

9 クラスタ

9.1 クラスタとは

Proxmox VE クラスタは、複数のノードを統合管理する仕組みです。Corosync と pmxcfs を使用して、設定の同期とクラスタ通信を行います。

9.1.1 クラスタの利点

- **一元管理**：すべてのノードを 1 つの Web UI から操作
- **ライブマイグレーション**：稼働中の VM を別ノードに無停止移動
- **高可用性（HA）**：ノード障害時に VM を自動フェイルオーバー
- **共有設定**：ユーザー、権限、ストレージ設定の共有

9.1.2 クラスタの要件

- 全ノードが同じバージョンの Proxmox VE
- 専用のクラスタネットワーク（推奨）
- 全ノード間で名前解決が可能
- 最低 3 ノード（クォーラム確保のため、2 ノードも可能だが非推奨）

9.2 クラスタの構築

9.2.1 最初のノードでクラスタを作成

```
# クラスタの作成
pvecm create <クラスタ名>

# 例
pvecm create my-cluster

# クラスタの状態確認
pvecm status
```

9.2.2 他のノードの参加

```
# 2 台目以降のノードで実行
pvecm add <最初のノードのIP>

# 例
pvecm add 192.168.1.100
```

9.2.3 クラスタの確認

```
# クラスタの状態
pvecm status

# ノード一覧
pvecm nodes

# Corosync の状態確認
systemctl status corosync
```

9.3 ライブマイグレーション

稼働中の VM を別ノードに無停止で移動できます。

9.3.1 前提条件

- 全ノードからアクセス可能な共有ストレージ（またはローカルストレージ間のコピー）
- 互換性のある CPU（同じベンダー推奨）
- 十分なネットワーク帯域

9.3.2 マイグレーションの実行

```
# オンラインマイグレーション（共有ストレージ使用時）
qm migrate <vmid> <移動先ノード> --online

# ローカルストレージ間のマイグレーション
qm migrate <vmid> <移動先ノード> --online --with-local-disks

# コンテナのマイグレーション
pct migrate <ctid> <移動先ノード> --online
```

9.4 高可用性（HA）

HA を設定すると、ノード障害時に VM/CT が自動的に別ノードで再起動されます。

9.4.1 HA の設定

```
# HA リソースの追加
ha-manager add vm:<vmid>

# 優先ノードの設定
ha-manager set vm:<vmid> --group <HAグループ>

# HA の状態確認
ha-manager status
```

9.4.2 HA グループ

HA グループでフェイルオーバー先のノードと優先順位を定義します。

```
# HA グループの作成（Web UI 推奨）
# Datacenter → HA → Groups → Create
```

9.4.3 フェンシング

HA が正しく機能するためには、フェンシング（障害ノードの強制停止）が重要です。Proxmox VE はデフォルトで watchdog ベースのフェンシングを使用します。

9.5 2 ノードクラスタ

2 ノード構成ではクォーラムの問題が発生するため、特別な設定が必要です。

```
# 2 ノードクラスタの場合、投票設定を変更
pvecm expected 1
```

ただし、2 ノード構成はスプリットブレインのリスクがあるため、 可能であれば 3 ノード以上を推奨します。QDevice を追加することで 2 ノードでも安全なクォーラムを実現できます。

```
# QDevice の設定（外部の軽量サーバーに配置）
pvecm qdevice setup <QDeviceサーバーIP>
```

9.6 クラスタからのノード削除

```
# 削除するノードで実行（クリーンな削除）
pvecm delnode <ノード名>

# 残りのノードで設定をクリーンアップ
rm -rf /etc/pve/nodes/<ノード名>
```

注意：ノード上の VM/CT は事前に他のノードにマイグレーションしてください。

10 Tips

10.1 Cloud-Init

Cloud-Init を使用すると、VM のデプロイ時にホスト名、ユーザー、SSH 鍵、ネットワーク設定を自動的に適用できます。

```
# Cloud-Init ドライブの追加
qm set <vmid> --ide2 local-lvm:cloudinit

# Cloud-Init 設定
qm set <vmid> --ciuser admin
qm set <vmid> --cipassword <パスワード>
qm set <vmid> --sshkeys ~/.ssh/id_rsa.pub
qm set <vmid> --ipconfig0 ip=192.168.1.50/24,gw=192.168.1.1
qm set <vmid> --nameserver 8.8.8.8
qm set <vmid> --searchdomain example.com
```

10.2 PCI パススルー

物理デバイス（GPU など）を VM に直接割り当てることができます。

10.2.1 IOMMU の有効化

```
# /etc/default/grub を編集
# Intel CPU の場合
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on"

# AMD CPU の場合
GRUB_CMDLINE_LINUX_DEFAULT="quiet amd_iommu=on"

# GRUB 更新
update-grub
reboot
```

10.2.2 VFIO モジュールの設定

```
# /etc/modules に追加
echo "vfio
vfio_iommu_type1
vfio_pci
vfio_virqfd" >> /etc/modules

# カーネルモジュールの更新
```

```
update-initramfs -u -k all
reboot
```

10.2.3 デバイスの割り当て

```
# IOMMU グループの確認
find /sys/kernel/iommu_groups/ -type l

# VM にデバイスを追加（Web UI または CLI）
qm set <vmid> --hostpci0 <デバイスID>
```

10.3 REST API の活用

Proxmox VE は完全な REST API を提供しており、自動化に活用できます。

```
# API トークンの作成（Web UI: Datacenter → Permissions → API Tokens）

# curl での API アクセス例
curl -k -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes

# VM 一覧の取得
curl -k -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes/<ノード>/qemu

# VM の起動
curl -k -X POST \
  -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes/<ノード>/qemu/<vmid>/status/start
```

10.4 Terraform との連携

Terraform の Proxmox プロバイダーを使用して、インフラをコードで管理できます。

```
terraform {
  required_providers {
    proxmox = {
      source  = "Telmate/proxmox"
      version = ">= 2.9.0"
    }
  }
}

provider "proxmox" {
  pm_api_url          = "https://192.168.1.100:8006/api2/json"
```

```
  pm_api_token_id     = "user@pam!terraform"
  pm_api_token_secret = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
  pm_tls_insecure     = true
}

resource "proxmox_vm_qemu" "example" {
  name        = "terraform-vm"
  target_node = "pve"
  clone       = "debian-template"
  cores       = 2
  memory      = 2048

  disk {
    size    = "32G"
    storage = "local-lvm"
  }

  network {
    model  = "virtio"
    bridge = "vmbr0"
  }
}
```

10.5 トラブルシューティング

10.5.1 よくある問題と対処法

10.5.1.1 Web UI にアクセスできない

```
# pveproxy サービスの確認
systemctl status pveproxy

# 再起動
systemctl restart pveproxy

# ファイアウォールの確認
iptables -L -n | grep 8006
```

10.5.1.2 VM が起動しない

```
# VM の設定確認
qm config <vmid>

# ログの確認
journalctl -u pve-qemu-server -f
cat /var/log/syslog | grep <vmid>
```

10.5.1.3 クラスタの同期問題

```
# クラスタファイルシステムの状態
pmxcfs -l

# Corosync の再起動
systemctl restart corosync
systemctl restart pve-cluster
```

10.5.1.4 ストレージの問題

```
# ZFS プールの状態
zpool status

# LVM の状態
lvs
pvs

# ストレージの再スキャン
pvesm scan <ストレージ種類> <サーバー>
```

10.6 便利なコマンド集

```
# ノードのリソース使用状況
pvesh get /nodes/<ノード名>/status

# 全 VM/CT のリソース情報
pvesh get /cluster/resources --type vm

# タスクログの確認
pvesh get /nodes/<ノード名>/tasks

# 設定ファイルのバックアップ
tar czf /root/pve-config-backup.tar.gz /etc/pve/

# SSL 証明書の更新（Let's Encrypt）
pvenode acme account register default <メール>
pvenode acme cert order
```